LECTURE 06

需求和架构（1）

06

让变化有地方可去：从真实需求到可追溯的历史

20 节 · 37 幅配图 · 5 道自测　回看课堂 ↗

复用 BaoCut fresh 原始转写，完整阅读19个分块、486条合并原句，保留完整96分50秒课程的时间导航；未逐秒听审。

原视频章节标记“引言（后半音频翻车）”；该段转写存在不流畅和术语错识，连续时间戳不证明音频质量。未凭讲义伪造口述补段。

官方现行 slides6.html 共47页及 lect6.md 完整阅读；其内容比课堂口述更丰富，新增细节均标讲义补充，未验证网页是否为录课当日版本。

讲义图来自官方网页完整页截图汇编，不是同秒视频帧；真实视频帧保留实际秒数。图片可点击放大。

桌面与窄屏浏览器质量复核由课程协调者补记；真实手机未实测。

阅读导引

先把握本课主线

01 AI 降低了写代码的成本，但**需求**、**产品与架构**决定代码朝哪里增长，以及下一次修改要付出多少代价。

02 真实教务系统、非递归汉诺塔、Excel 二维码和事件溯源共同说明：**选择表示方式**，就是为复杂性与变化安排位置。

03 本课特别保留需求调研→只凭规格实现原型的现场链条，以及“缺课次数”的**校验争议**、课程替代、组织改名和历史重放的边界。

第 01 节

00:00:00—00:03:24 · 课堂时间

从意图到实现：错误会沿依赖传播

> **关键判断 · 整理者提炼**
>
> 需求、设计、实现与测试互相依赖；可追溯性帮助识别上游改动会使哪些下游产物失效。

本课接续上一讲的 Intent → Specification → Implementation：客户的意图需要变成明确规约，再成为实现。实现之后仍有**维护阶段**，因此不能把软件生命周期理解成“代码写完就结束”。需求认识、设计决定、模块、函数、测试都互相依赖，构成**数据依赖图**。

1 / 47

资料图 · 第 1 页 回顾：Intent—Spec—Impl Gap、瀑布模型与 Data–Dependency Graph。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 1 · 原始讲义 · 回顾：Intent—Spec—Impl Gap、瀑布模型与 Data–Dependency Graph。

如果到实现阶段才发现某个需求或设计错误，修复会反向追到上游，再让依赖它的下游代码与测试失效。Royce 对简单瀑布式过程的批评，正是看到这种**返工风险**。软件工程因此既包括如何沟通、组织人和检查需求，也包括测试、分析等技术方法。

可追溯性（traceability）的理想是：每个测试用例或代码片段，都能追到它依据的设计或需求；修改一个设计时，可以知道哪些产物需要更新。UML 是试图用结构化、具有语义的共同语言对齐不同环节的尝试。

第 02 节

00:03:24—00:06:03 · 课堂时间

软件不软：微小偏差为什么能摧毁系统

> **关键判断 · 整理者提炼**
>
> 软件能修改代码，却仍可能因微小表示错误或业务变化而大幅改变行为。

物理椅背受力后能回弹，软件却是遵从**形式语义**的逻辑制品。一个很小的**表示错误**可能完全改变执行过程，“能改代码”不等于“能轻松承受变化”。

资料图 · 第 2 页 官方讲义：软件工程的基本观察——软件不软。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 2 · 原始讲义 · 官方讲义：软件工程的基本观察——软件不软。

讲师用硬件错误作极端例子：本来读入 N=1，若无符号整数高位被翻转，循环可能尝试读取巨大数量的数据，出现等待、内存错误或崩溃。随后提到数据中心处理器的**静默错误**，意在说明软件正确性还依赖运行环境的假设。

> **整理者说明**
> 这些硬件现象是讲师的背景例证；本笔记没有独立审阅硬件错误研究，也不把“一位翻转必然使所有程序错误”当作严格定理。

更常见的困难来自客户需求：软件既要有精确的逻辑，又要承接不断变化的**业务世界**。教务系统将作为本课贯穿案例。

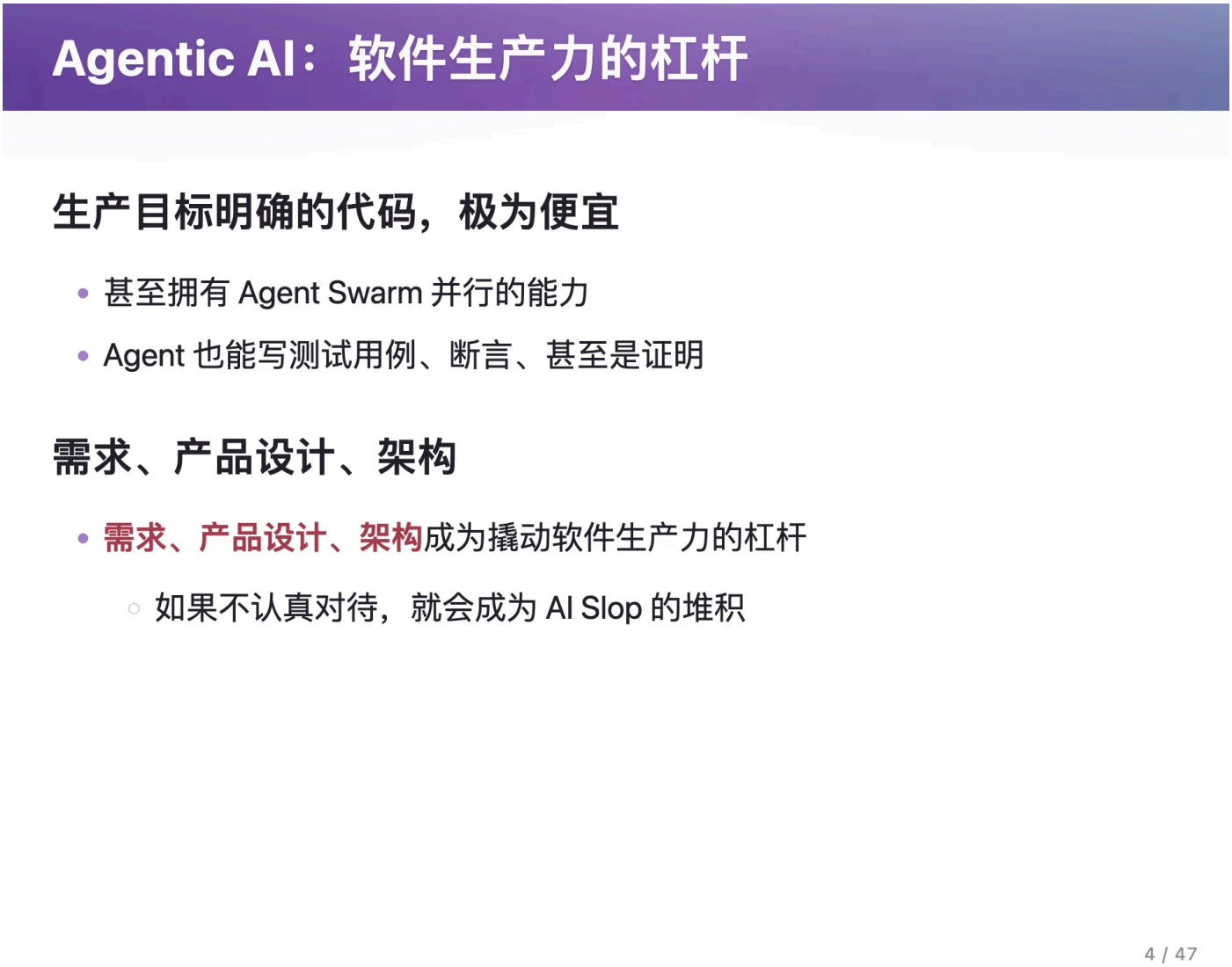
第 03 节

00:06:03—00:09:53 · 课堂时间

代码便宜以后，需求、产品与架构成为杠杆

> **关键判断 · 整理者提炼**
> 当生成代码更便宜，需求、产品和架构更需要先明确目标、边界与验证方式。

传统学生往往从语言与编码出发，逐步理解设计和需求；没有实现经验，很难判断一项要求的**成本与可行性**。但决定软件从零开始走向哪里的关键，通常发生在**需求、产品和架构**阶段。



资料图 · 第 4 页 Agentic AI：目标明确的代码生产便宜，需求、产品设计与架构影响后续软件生产。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 4 · 原始讲义 · Agentic AI：目标明确的代码生产便宜，需求、产品设计与架构影响后续软件生产。

讲师指出，这些问题难做统一的定量研究，研究中容易转向缺陷预测、代码分析等更易量化对象。这是课堂观点与调侃，不能理解为所有软件工程研究都没有实践价值。

当**目标和边界**足够清楚，AI 可以生成实现、测试、断言，多个 Agent 可以并行工作。讲师称自己在某些边界明确、验证方式清楚的任务上已经不逐行阅读实现。这里的前提是边界与 **validation**，并非对任意生成代码的无条件信任。

第 04 节

00:09:53—00:15:03 · 课堂时间

MVP、早期决定与不断进入上下文的技术债

> **关键判断 · 整理者提炼**
>
> MVP 要走通关键路径，同时识别会持续进入后续上下文的早期决定和设计负担。

MVP 选择最重要的功能，从意图到规约再到实现走通一个**纵向切片**。不要过早打磨依赖尚未稳定设计的细节：例如按钮样式依赖整体界面，界面又依赖功能安排，上游变化会让此前打磨被丢弃。

同一份功能需求，仅增加“CLI”或“client-server”等架构提示，就可能生成完全不同的项目：前者会引入命令行解析，后者会先考虑服务、数据库和存储。第一个文件一旦写入，就会进入 Agent 的**后续上下文**，影响以后每次修改。

早期决定未必错误，但会构成以后必须承接的约束。个人工具的命令行偏好、隐含知识与操作习惯，不能直接搬给普通用户；需求、产品设计和架构共同决定软件走向。

引言末尾有一次实际设备故障：系统界面和 OBS 推流仍在工作，鼠标键盘却不能使用。讲师转向日志诊断，并提醒生成软件若没有足够日志，故障后很难解释发生过什么；**日志缺失**也是会留下来的设计负担。

> **整理者说明**
> 视频原始章节标为“引言（后半音频翻车）”。现有转写仍覆盖这一段，但这不能证明每句听辨准确；未将讲义补充伪装为修复后的口述。音频诊断范围与限制详见本课 audit.json。

第 05 节

00:15:03—00:23:20 · 课堂时间

真实教务系统演示：需求远不止几张表

> **关键判断 · 整理者提炼**
> 教务系统的每个功能都带着身份、时间与业务条件，页面能运行只是交付的一部分。

讲师登录现有教务系统，先试用隐藏的 AI 服务检索：询问教室灯具问题应联系谁，又问“我现在在上什么课”。后者返回**教师课表入口**，需要再进入本科教师身份查看课表；课堂没有证明 AI 已完整理解个人教学上下文。

随后依次浏览教学任务、**教学周历**、历史授课学期、期末考试签到表、教材编辑、大纲和助教等页面。每一个按钮都伴随条件：考试未发布时不能打印签到表；助教只能在维护时间内申请；**多身份**、**多学期**必须可选择并明确显示。

课堂出现教材编辑迟迟未响应、随后页面刷新，以及慕课助教列表显示异常等现象。这些是现场观察，未定位具体后端故障原因。教学班与慕课助教是需求逐年叠加的例子：一种新授课方式出现后，还会带来新角色和新管理规则。

真实交付同时面对功能与**非功能要求**，并且需要承担合同责任。看似能实现页面，不等于理解整个业务或承受长期维护。讲师关于“把预算换成 token 可能更便宜”的说法是课堂设想，没有成本核算支撑。

第 06 节

00:23:20—00:30:12 · 课堂时间

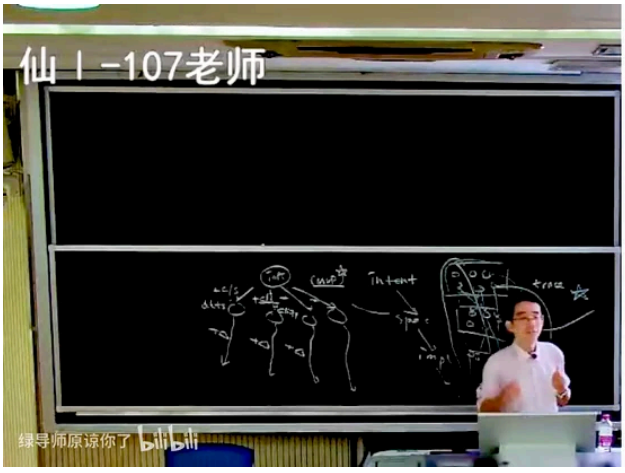
让 Agent 逆向整理需求：编号、证据与验收

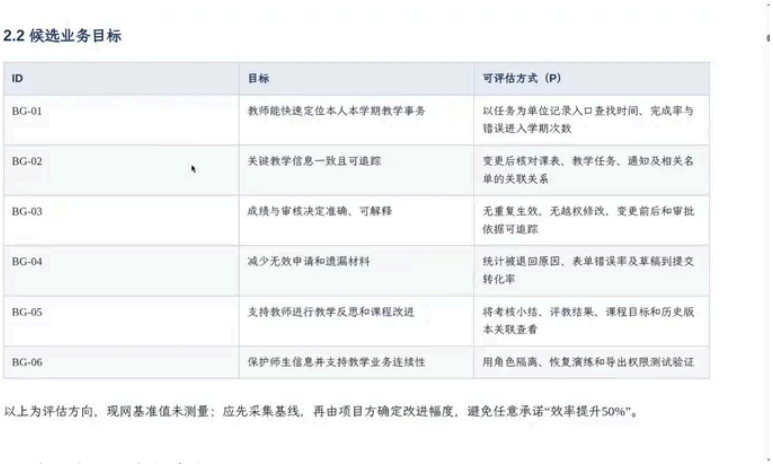
> **关键判断 · 整理者提炼**

> 需求编号、业务证据、共同术语和验收场景，让需求说明成为可分工、可检查的依据。

讲师让 Agent 探索现有系统中教师相关功能，形成需求说明书。范围明确是教师端相关功能，不能把它当作全校所有业务的完整规格。说明书包含调研方法和**证据边界**、业务目标、成功标准、角色、用例规约、功能目录、**验收场景**、术语与配套材料。

一个需求点不只写一句自然语言，还通过**稳定编号**关联参与者和业务目标。这使“教学班与学生管理有关”成为可检查的关系，帮助实现 **traceability**。术语表则让客户与开发者在词义不清时有共同参照。





2.2 候选业务目标

ID	目标	可评估方式（P）
BG-01	教师能快速定位本人本学期教学事务	以任务为单位记录入口查找时间、完成率与错误进入学期次数
BG-02	关键教学信息一致且可追踪	变更后核对课表、教学任务、通知及相关名单的关联关系
BG-03	成绩与审核决定准确、可解释	无重复生效、无越权修改，变更前后和审批依据可追踪
BG-04	减少无效申请和遗漏材料	统计被退回原因、表单错误率及草稿到提交转化率
BG-05	支持教师进行教学反思和课程改进	将考核小结、评教结果、课程目标和历史版本关联查看
BG-06	保护师生信息并支持教学业务连续性	用角色隔离、恢复演练和导出权限测试验证

以上为评估方向，现网基准值未测量；应先采集基线，再由项目方确定改进幅度，避免任意承诺“效率提升50%”。

视频原帧 现场需求说明书：用编号关联需求与角色/目标；此图是 25:40 的原视频帧。

视频原帧 00:25:40 · 现场需求说明书：用编号关联需求与角色/目标；此图是 25:40 的原视频帧。

课堂逐项读到服务分类与检索、多种已分配身份下的身份选择、当前身份显示、学年学期或业务批次及授权范围内切换。它们看似繁琐，却决定功能到底有没有实现。

有结构的需求清单与**验收场景**可以分派给不同团队，并据此跟踪进度。讲师用计算机系统基础课程的 NEMU 文档类比：OJ 的简单输入输出，与模拟器、调试器、指令行为的长规格，复杂度不同，但都在训练理解和实现规约。

第 07 节

00:30:12—00:32:51 · 课堂时间

只凭需求生成原型：成功路径与验证边界

> **关键判断 · 整理者提炼**
>
> 通用数值上限与实际授课次数的业务约束是两回事，原型成功不能替代完整验证。

演示中的第二个 Agent 被要求不看原系统，仅根据需求说明书生成一个假的教务系统。讲师打开生成页面，查看教学任务、教材记录和学生名单，并编辑教材后保存。它与原系统在功能结构上相似，说明清晰规格能显著约束生成结果。

讲师先尝试输入 –1，界面不允许；再输入极大的正数，31:52画面弹出“Value must be less than or equal to 100.”，证明存在**数值上限100**。随后改成100，31:55学生列表显示“100次”和“缺课记录已更新”。**100仍可能超过**本学期全部课次，因此讲师说“他没有阻止我”指的是业务上不合理的次数仍可保存。

视频原帧 31:52 原视频：超大数被拒，浏览器显示必须小于或等于100。

视频原帧 00:31:52 · 31:52 原视频：超大数被拒，浏览器显示必须小于或等于100。

视频原帧 31:55 原视频：改成100后保存成功。通用数值上限不等于符合本学期实际课次。

视频原帧 00:31:55 · 31:55 原视频：改成100后保存成功。通用数值上限不等于符合本学期实际课次。

后面讲师说大数被拒绝，与画面并不矛盾：极大数被拒和100被保存是两个状态。逐帧核对后可区分“有一个通用数值上限”与“依据实际授课次数验证**业务合理性**”。画面证明了**前端提示和保存结果**，不能据此声称后端绕过测试、权限、并发和全部例外均已通过。

第 08 节

00:32:51—00:36:19 · 课堂时间

同理心与领域知识：需求为何永远不能一次说清

> **关键判断 · 整理者提炼**
>
> 需求需要理解真实使用者与规则演变，课程拆分等变化会暴露第一版模型的表达边界。

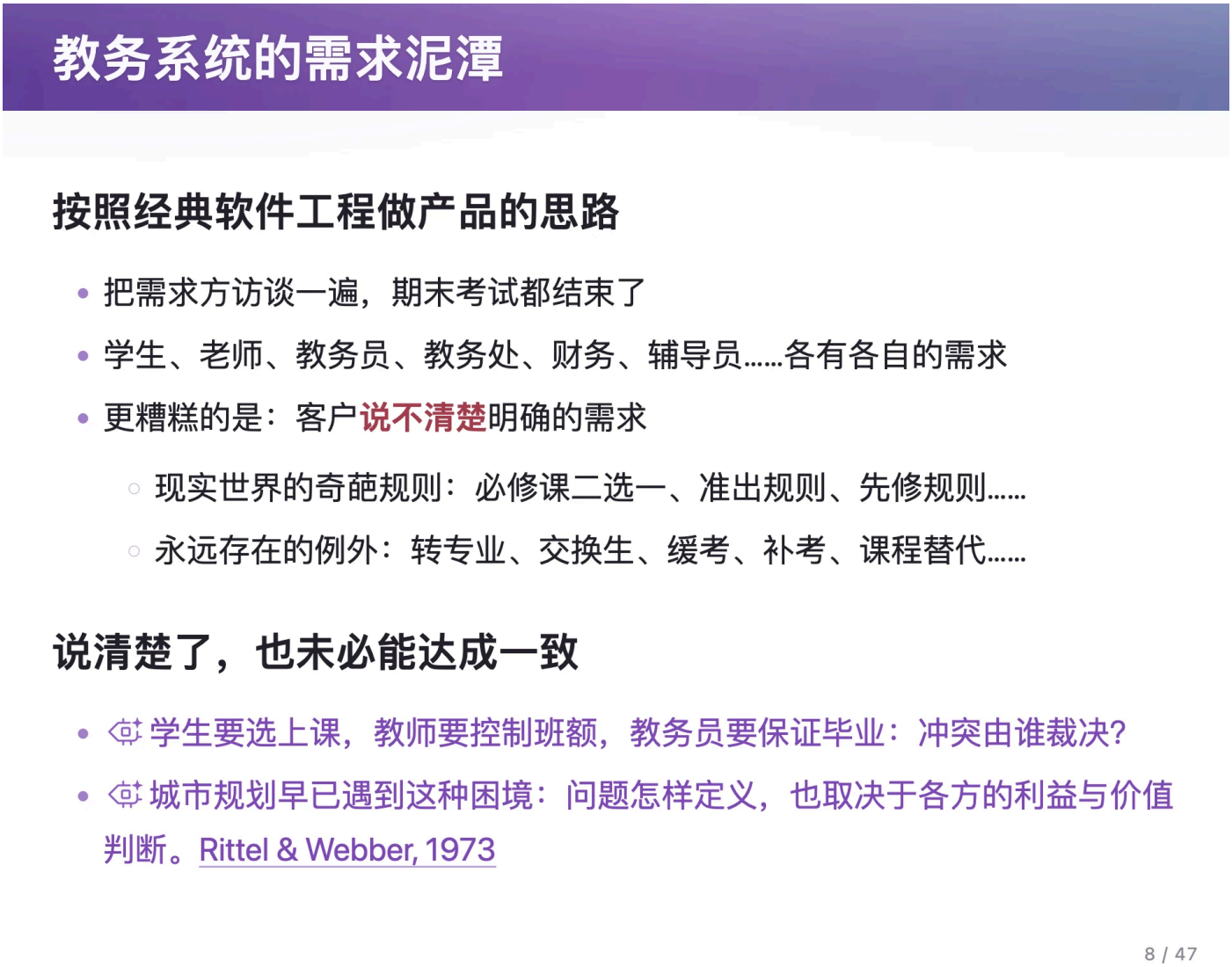
从入学到毕业的业务链，不只包含学生、课程与成绩，还包括**权限**、**规则版本**、数据同步与操作留痕。需求分析要不断和客户沟通，连“学期选择放哪里”“哪个位置有修改按钮”也需要对齐。

教务系统：总需求分解，从入学到毕业

资料图 · 第 7 页 官方讲义需求分解图：从入学到毕业的参与者、过程与支撑能力。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 7 · 原始讲义 · 官方讲义需求分解图：从入学到毕业的参与者、过程与支撑能力。

设计者不是用户本身。讲师以熟悉程序的年轻开发者和即将退休的教务人员作对照：后者可能更在意字体、按钮大小与少误点，而非最新数据库或最优性能。**行业经验**的价值，是知道学位授予流程和真实使用者如何工作。



资料图 · 第 8 页 需求泥潭：多方目标、规则和例外；讲义补充了利益冲突与裁决问题。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 8 · 原始讲义 · 需求泥潭：多方目标、规则和例外；讲义补充了利益冲突与裁决问题。

实现第一版后，系统就背上了已有决定。客户难以一次列出所有条件，规则还会改变。课堂的真实引子是离散数学课程被拆成两门：旧培养方案、准出条件与保研要求怎样与新课程衔接，原来的**单课程下拉框**已经无法自然表达。

第 09 节

00:36:19—00:40:51 · 课堂时间

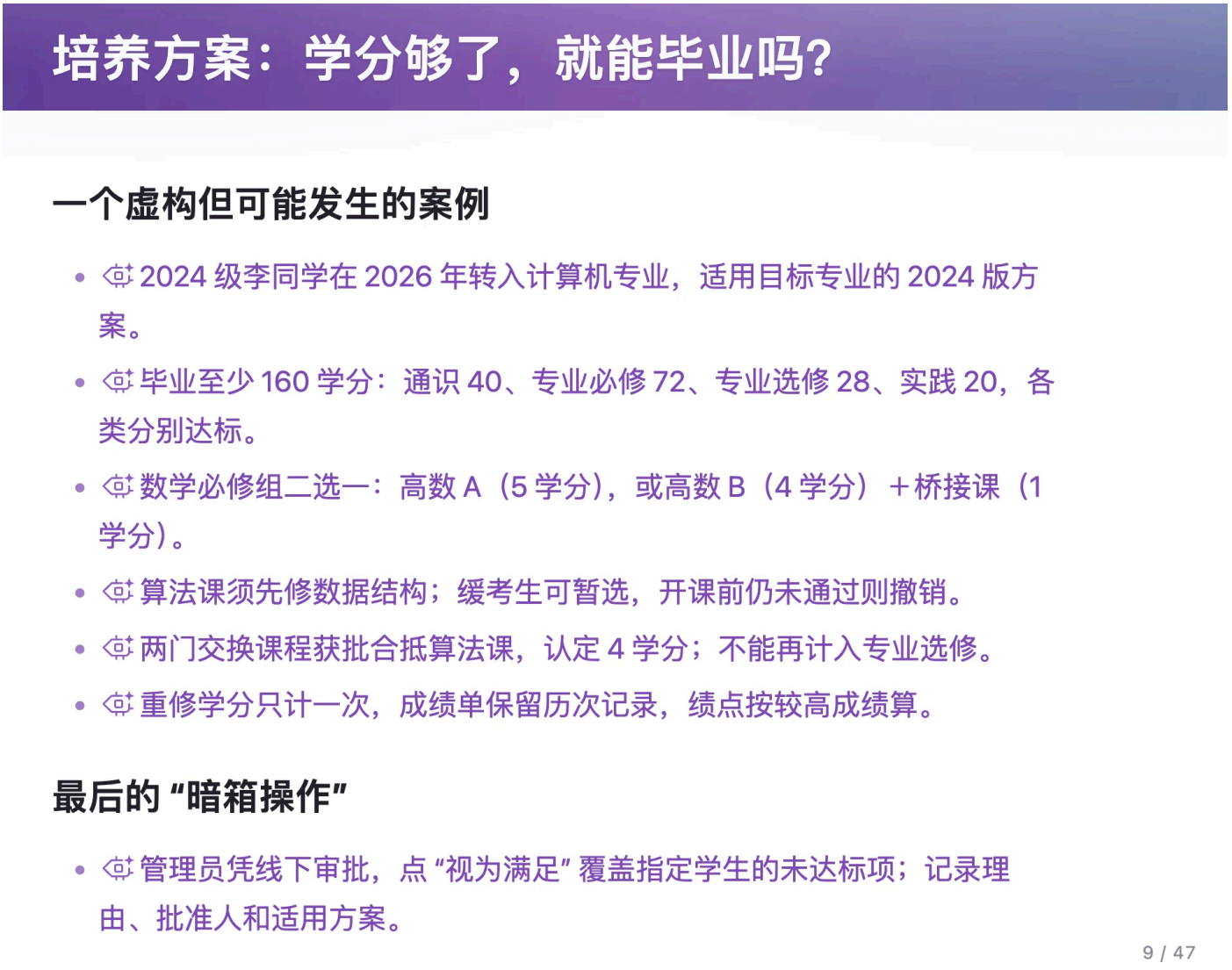
课程拆分、替代与人工豁免：局部变更穿透全系统

> **关键判断 · 整理者提炼**
>
> 局部课程变更会穿透数据、算法与历史口径；人工批准也应保留适用范围和依据。

原始毕业要求可能只允许“增加一门必修课程”。课程拆分后，规则却需要表达旧课程或新课程组合的**替代关系**。增加一个“或”不是单纯修改下拉框：数据表示、审核算法、**历史学生适用条件**和相关界面都可能被影响。

再叠加转专业、交换、重修与抵扣，问题更复杂。另一个院系的一门离散数学，究竟抵新方案的一门还是两门？临近毕业的学生碰到课程只在另一个学期开设，不能等下一年，又该怎样处理？

> **整理者说明**
> 官方讲义第9页的 160 学分、40/72/28/20 分类、高数A或B+桥接、交换课合抵与绩点规则是明确标为虚构的示例，不是南京大学实际政策。课堂没有逐条口述这些数字，故以下作为讲义补充保留。



资料图 · 第 9 页 讲义补充｜虚构培养方案案例。160 学分及各分类数字不可当作学校政策。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 9 · 原始讲义 · 讲义补充｜虚构培养方案案例。160 学分及各分类数字不可当作学校政策。

现实常出现**线下批准**：教学负责人认为旧课程可以替代新要求，而系统没有表达能力，于是联系工程师直接把毕业条件改为满足。业务上可能有合理解释，系统里却只剩一个结果，解释与依据留在系统外。

> **整理者说明**
> 讲义补充：人工“视为满足”也应有需求——**批准人、理由**、适用学生、豁免项目与方案版本，而不是只有一个无来源的布尔值。

缺课次数 −1 与“没有银弹”

> **关键判断 · 整理者提炼**
> 局部数据假设可以影响多个页面；快速验证 MVP 不能消除长期业务的依赖复杂性。

讲师回忆首次使用教师端系统时，**绕过前端校验**，把一位学生缺课次数从 0 改成 −1；随后所有能选出该学生选课记录的页面都返回 500。其后联系教务，次日恢复，但具体修复方式未知。

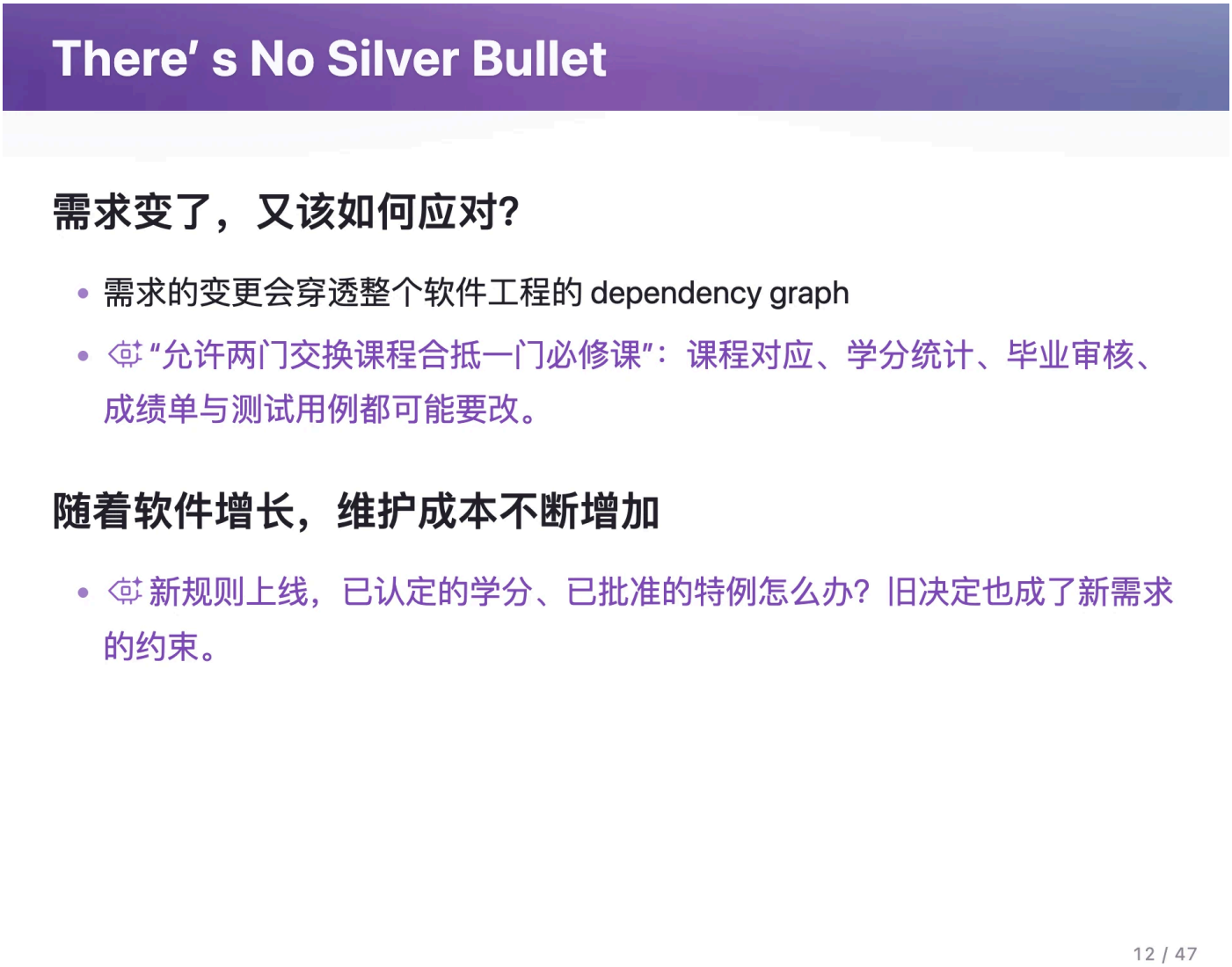
10 / 47

资料图 · 第 10 页 旧教务系统的 −1 案例：局部异常沿依赖传播。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 10 · 原始讲义 · 旧教务系统的 −1 案例：局部异常沿依赖传播。

这个例子说明局部数据假设可能被系统多个地方**依赖**。它不证明具体错误必然来自某种数据库类型，也不证明只加前端限制就能解决。现场生成原型中的验证表现，与这段旧系统经历需要分开。

MVP 能快速暴露错误、降低早期推倒重做成本，但也会造成驾驭复杂系统的错觉。上线后系统必须承接现实业务、既有数据与历史决定，不能像课程项目那样随时清零。需求沿**依赖**图传播，持续变化使维护成本增长；“没有银弹”是没有一招彻底消灭这种复杂性。



资料图 · 第 12 页 官方讲义：需求变更可以同时影响课程对应、学分、审核、成绩单与测试。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 12 · 原始讲义 · 官方讲义：需求变更可以同时影响课程对应、学分、审核、成绩单与测试。

第 11 节

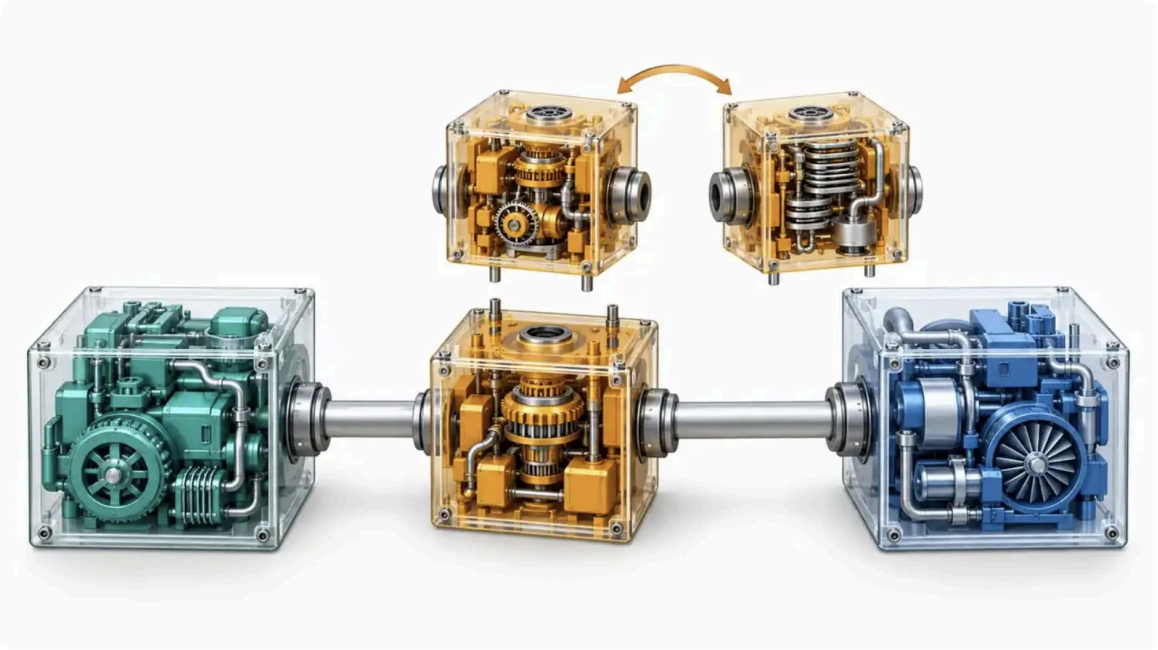
00:44:29—00:51:26 · 课堂时间

架构从小程序开始：把复杂性和变化隔开

> **关键判断 · 整理者提炼**
>
> 架构通过明确的数据与责任边界隔离变化，拆成多个函数本身并不足够。

架构要回答：最小的软件如何在未来膨胀和变化时仍有**韧性**？“先把所有需求塞进数据库”能很快跑起来，但如果概念之间没有合理分离，随后会陷入维护泥潭。架构并不只属于大系统。

好的架构：复杂性有归处，变化有边界



资料图 · 第 15 页 官方图示：复杂性有归处，变化有边界。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 15 · 原始讲义 · 官方图示：复杂性有归处，变化有边界。

最小的 main 也可以分为 read_input、parse_input、process_input、write_output。命令行的 argc/argv 包含长短参数和约定，用 **getopt** 解析后得到可供业务处理的状态，就是把字符串格式复杂性关在边界里的例子。

再小的代码也有架构

甚至最小的 “状态机” 就是架构

```
int main() {
    read_input();
    parse_input();
    process_input();
    write_output();
}
```

编程语言与架构

- 你学习的语法、库函数……都可能和架构有千丝万缕的联系
- 所以我不喜欢老登那种 “语言律师” 的授课方式
  - 语义的价值，是把设计意图变成约束：谁能改数据，谁负责资源，谁可以依赖谁。
  - 回看这四个函数：数据怎样穿过各阶段？换一种输入格式，需要改到哪里？

16 / 47

资料图 · 第 16 页 小程序四阶段结构；数据如何穿过边界比函数数量更关键。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 16 · 原始讲义 · 小程序四阶段结构；数据如何穿过边界比函数数量更关键。

> **整理者说明**
> 讲义补充：仅切成四个函数不自动形成架构边界；仍需明确**数据所有权**、资源责任和依赖方向，避免共享全局状态把函数重新耦合起来。

每个模块里都可以讨论 intent、specification 和 implementation。学习库函数与语言特性不应停在语法用法，而应继续问“为什么这样组织、它隔离了什么变化”。把复杂输入转换为明确**中间状态**，这个模式还会在二维码编译器里再次出现。

第 12 节

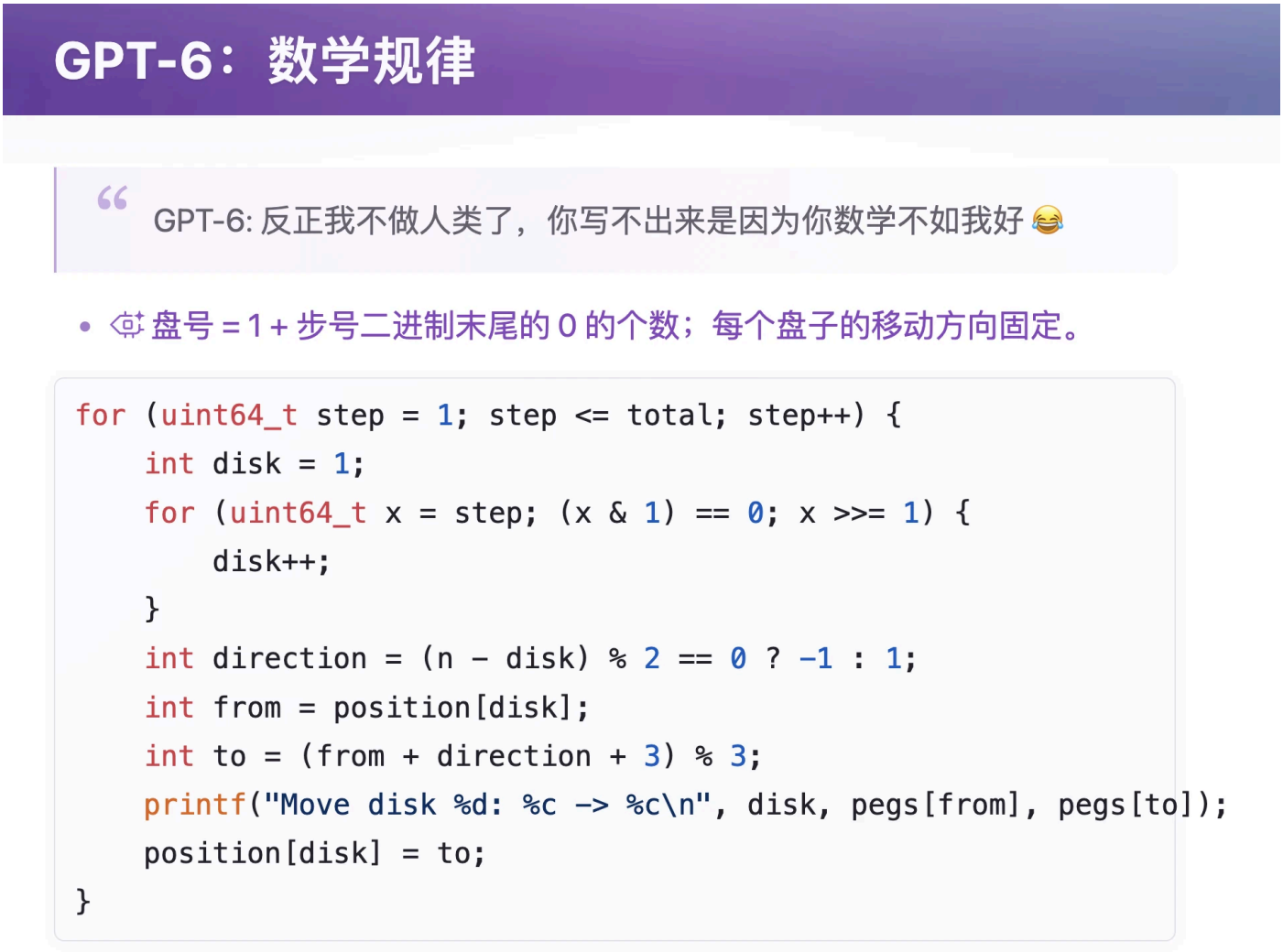
00:51:26—00:57:58 · 课堂时间

非递归汉诺塔：数学规律、任务栈与抽象机器

> **关键判断 · 整理者提炼**
> 数学规律、任务栈与抽象机器把复杂性放在不同层次，选择表示要看推理和推广的需要。

同一个非递归汉诺塔需求，能用完全不同的表示实现。讲师用几种模型与自己的实现对照；这些是课堂个案，没有统一模型评测设计，不能外推为模型普遍排名。

第一种用数学规律：第 step 步移动的盘号等于 1 加 step 二进制尾随零的个数，每个盘子的方向固定。复杂性被移到了规律及其证明中，代码很短，却需要理解这些**不变量**；总移动次数仍是 2^n−1。



资料图 · 第 18 页 数学规律方案：尾随零决定盘号，奇偶关系决定方向。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 18 · 原始讲义 · 数学规律方案：尾随零决定盘号，奇偶关系决定方向。

> **整理者说明**
> 官方讲义为 uint64_t 写法；完整讲义文字限定 1≤n<64，并要求足够的 position 数组。截取循环本身不是可直接编译的完整程序。

> **案例回扣** 整理者依据官方代码验证：n=3 时移动盘号为 1,2,1,3,1,2,1；对 n=1..10 模拟柱栈、检查每次合法性并与递归序列比较，全部一致。此为独立小算例检查，不是对原课堂完整代码工程的测试。

第二种用**任务栈**：保存尚未完成的搬盘任务，每次弹出栈顶；n=1 直接移动，否则拆成三项任务。由于后进先出，压入顺序必须与期望执行顺序相反。这保留递归的任务结构，只把隐式调用栈改成显式数据。

DeepSeek：任务栈

> DeepSeek: 人类语料里似乎让我这么干

```
push(&s, n, A, B, C);
while (!isEmpty(&s)) {
    Task t = pop(&s);
    if (t.n == 1) {
        printf("%c -> %c\n", t.src, t.dst);
    } else {
        push(&s, t.n - 1, t.aux, t.src, t.dst);
        push(&s, 1, t.src, t.aux, t.dst);
        push(&s, t.n - 1, t.src, t.dst, t.aux);
    }
}
```


资料图 · 第 19 页 任务栈方案：三次 push 采用执行顺序的反序。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 19 · 原始讲义 · 任务栈方案：三次 push 采用执行顺序的反序。

第三种模拟抽象机器：每个栈帧保存 n、三根柱子角色与**程序计数器** pc；调用压栈、返回出栈，以**小步状态转移**模拟 C 程序的操作语义。它对汉诺塔可能显得过重，却能推广到相互递归等更一般情形。

绿导师：抽象机器

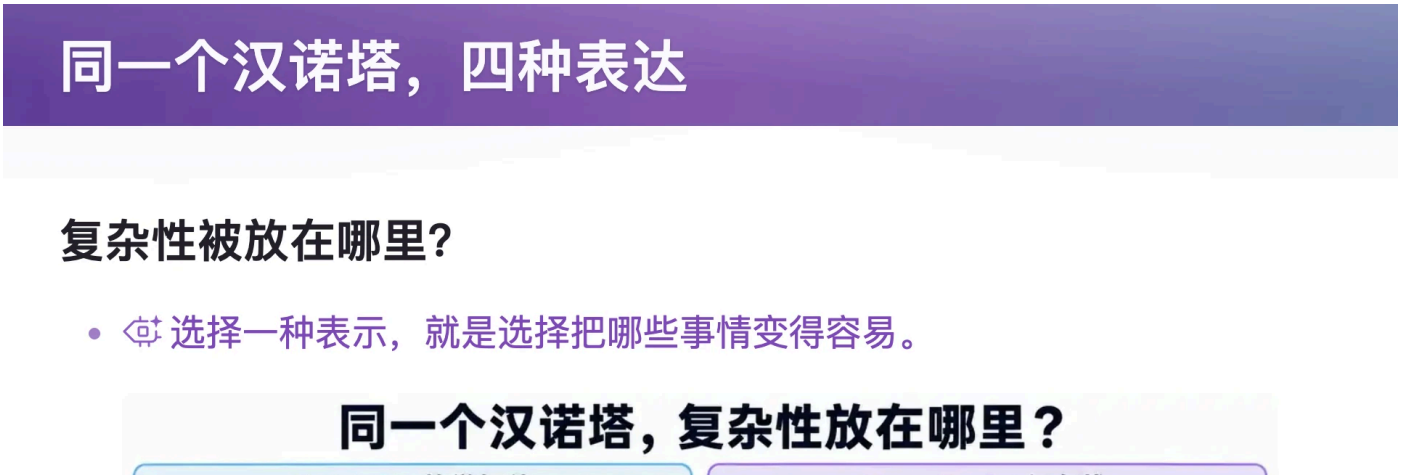
> 绿导师：这是理解 C 语言“形式语义”的好机会

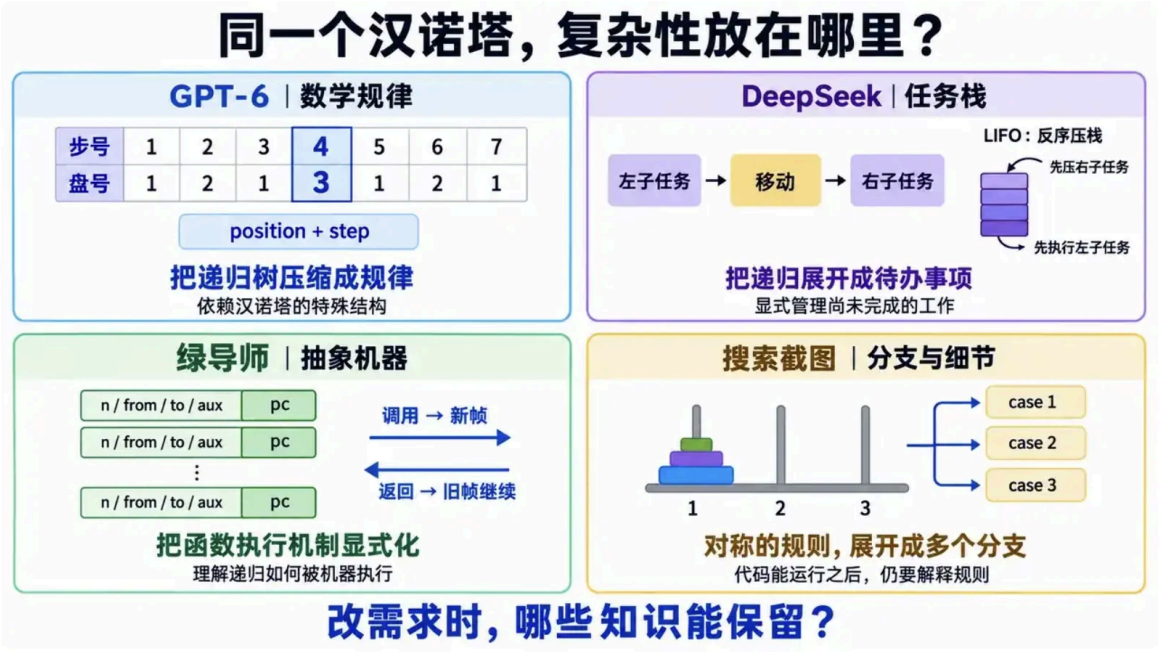
```
#define call(...) (*(++top) = (Frame){.pc = 0, __VA_ARGS__})
#define ret() (--top)
call(n, from, to, aux);
while (top >= stk) {
    Frame *f = top;
    int next_pc = f->pc + 1, n = f->n, from = f->from, to = f->to, aux = f->aux;
    switch (f->pc) {
        case 0: if (n == 1) { printf("%c -> %c\n", from, to); ret(); } break;
        case 1: call(n - 1, from, aux, to); break;
        case 2: call(1, from, to); break;
        case 3: call(n - 1, aux, to, from); break;
        case 4: ret(); break;
        default: assert(0);
    }
    f->pc = next_pc;
}
```


资料图 · 第 20 页 抽象机器方案：Frame、pc、call 与 ret 显式表示递归执行。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 20 · 原始讲义 · 抽象机器方案：Frame、pc、call 与 ret 显式表示递归执行。

讲师还展示搜索得到的复杂代码作为反例。重点不是一味追求行数少，而是选择何种表示能让需求更清楚、更好推理。数学结构、任务分解和执行机器，分别把复杂性放在不同层次。





资料图 · 第 22 页 同一个问题的不同表达：选择表示就是选择复杂性的位置。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 22 · 原始讲义 · 同一个问题的不同表达：选择表示就是选择复杂性的位置。

第 13 节

00:57:58—01:04:45 · 课堂时间

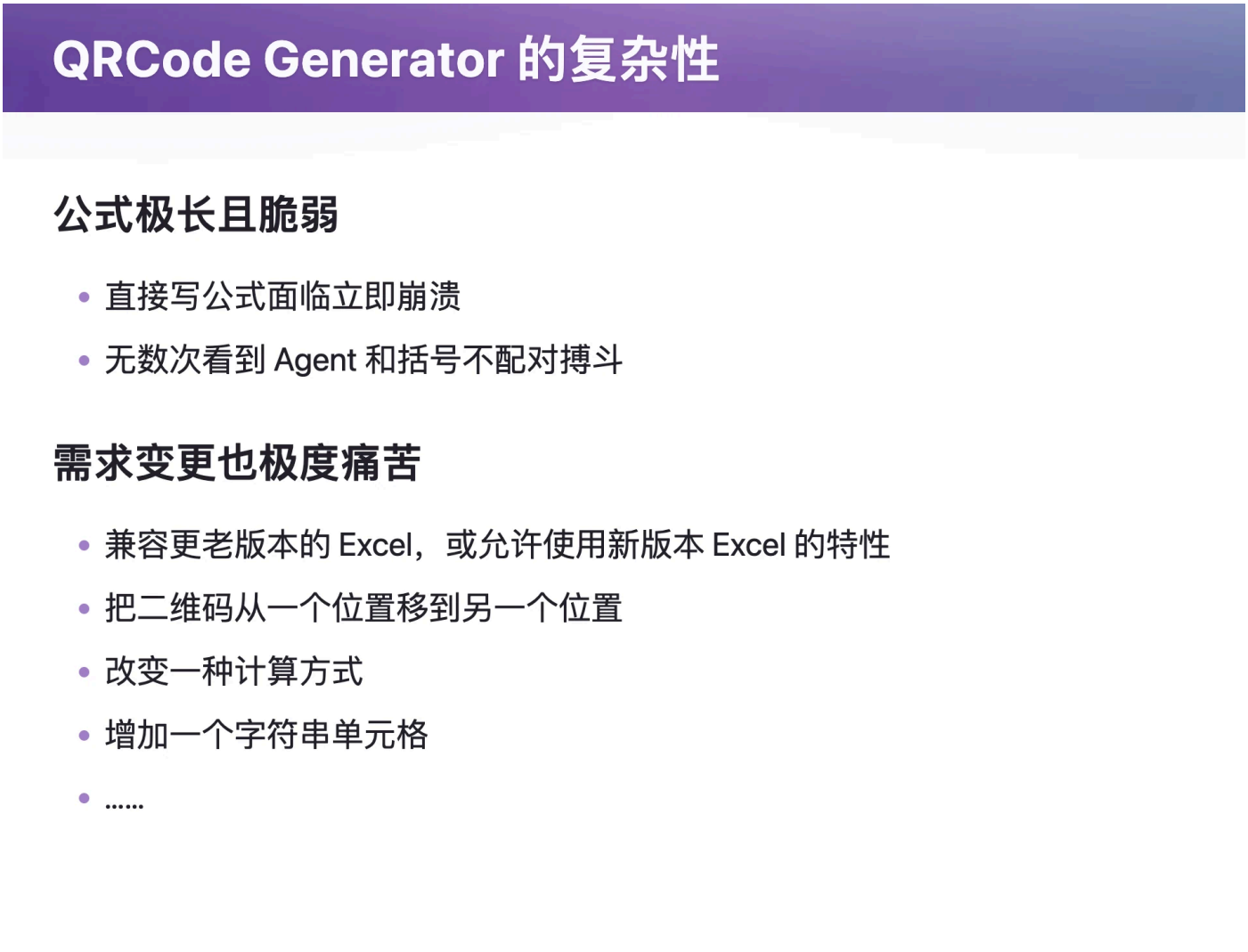
Excel 二维码：明确需求也隐藏着演化风险

> **关键判断 · 整理者提炼**
>
> 二维码功能即使明确，公式布局、版本兼容和新增输入仍需要各自可修改的位置。

需求很明确：用户修改一个输入单元格，工作表公式重新生成可扫描二维码。但直接拼出**大量公式**，会把括号、引用、算法步骤与布局绑在一起；模型反复修括号只是表面症状。

版本兼容也是需求：只给自己用可以采用新函数，要发给安装旧版 Excel 的用户就要考虑旧功能。课堂以 VLOOKUP/XLOOKUP 类比功能演化，未精确确认发布时间，本笔记不把课堂口述年份当作软件版本事实。

把二维码整体移一格、增加输入、改变计算方法，都可能让**硬编码公式**大范围失效。两千行小程序可以重写，但它一旦嵌进更大系统，关联就使重做昂贵。好的架构应让这些变化分别有**明确位置**。



资料图 · 第 24 页 官方讲义：长公式、版本兼容、布局移动与新增输入等变化。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 24 · 原始讲义 · 官方讲义：长公式、版本兼容、布局移动与新增输入等变化。

第 14 节

01:04:45—01:11:08 · 课堂时间

中间表示：先正确表达计算，再生成目标公式

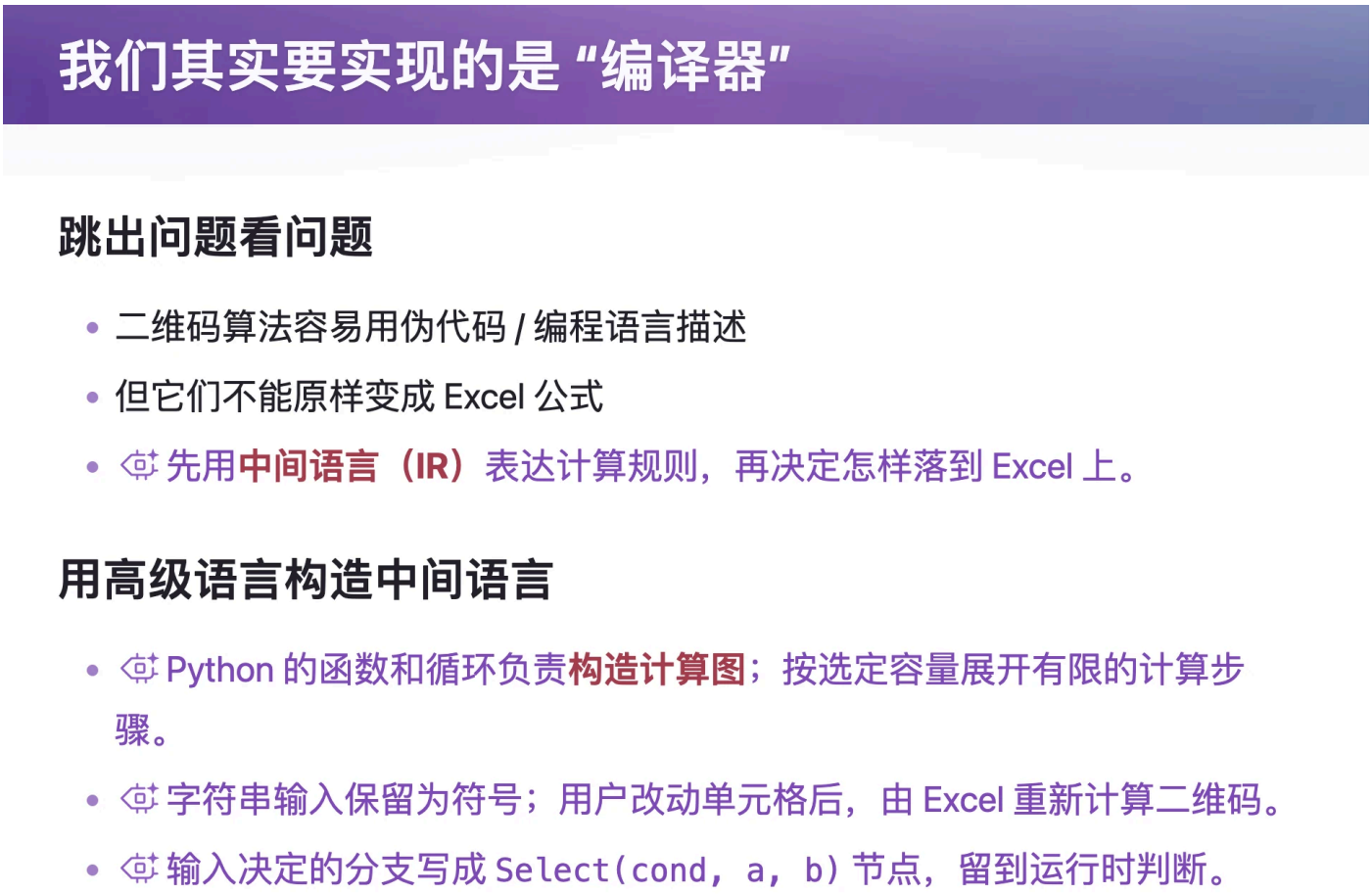
> **关键判断 · 整理者提炼**
>
> 中间表示分开算法与目标公式；保留运行时输入，并用独立验证防止两个后端共享错误。

二维码可以用简洁的伪代码描述，困难在于把这种计算搬到 Excel。讲师现场让 Agent 写算法概述，展示版本、纠错等级、掩码、数据编码与纠错计算。课堂这一处口述细节不足以重建完整二维码标准，因此不将零散系数或码字数作为通用参数。

> **整理者说明**
> 课堂说“没有 Python 到 Excel 编译器”是这一案例的任务语境，不作为全世界工具不存在的断言。

解法是在 Python 与 Excel 之间设计**中间语言**。比如异或、索引、字符串取值等运算成为**表达式树**节点；每个节点有清楚语义，并各有 Python 和 Excel 的翻译规则。

> **整理者说明**
> 讲义补充：生成时可用 Python 循环展开有限计算图；用户字符串应保留为**符号输入**，输入决定的分支须形成 Select 节点，留给运行时重算。若生成时直接固定输入，得到的只是一张固定二维码。



25 / 47

资料图 · 第 25 页 讲义补充的关键边界：构图时与运行时，符号输入与 Select。

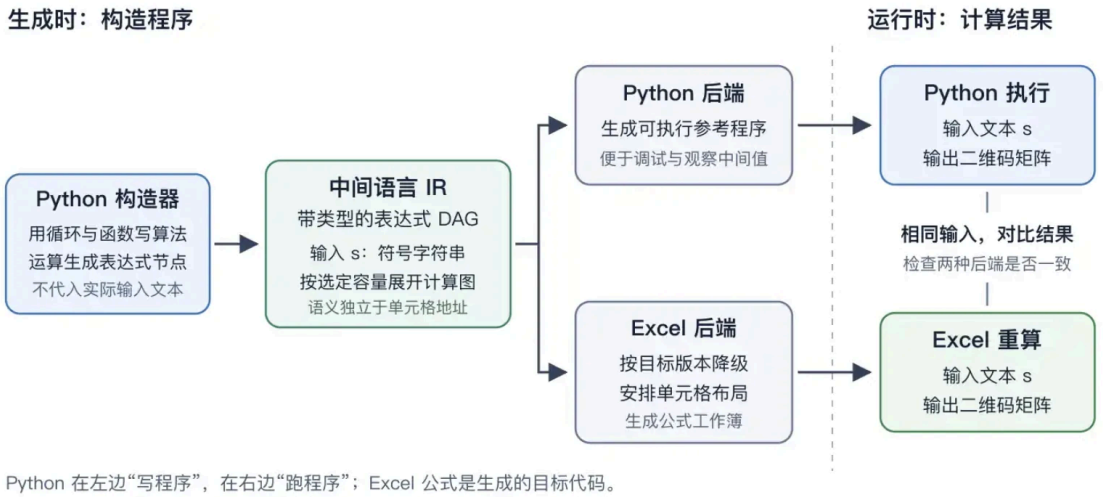
官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 25 · 原始讲义 · 讲义补充的关键边界：构图时与运行时，符号输入与 Select。

同一计算图先生成 Python 便于调试和验证，确认算法后再生成 Excel 交付。**表达式树**递归输出时，生成器统一处理子表达式、括号与单元格引用；算法不用逐处硬编码地址。

一份中间语言，两种执行方式

构造一次，分别生成

- 同一个计算图，既能生成便于调试的 Python 程序，也能生成 Excel 公式。



Python 在左边“写程序”，在右边“跑程序”；Excel 公式是生成的目标代码。

资料图 · 第 26 页 一份中间语言，两种执行方式：调试验证与 Excel 交付。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 26 · 原始讲义 · 一份中间语言，两种执行方式：调试验证与 Excel 交付。

> **整理者说明**
>
> 讲义补充：两个后端可能共享同一算法错误。除比较 Python 与 Excel 实际重算结果，还需用**独立实现或解码器**校验；逐像素比较前须固定版本、纠错等级、分段与掩码。

资料图 · 第 28 页 讲义补充：IR 类型/容量/编码语义与独立验证，避免两后端同错。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 28 · 原始讲义 · 讲义补充：IR 类型/容量/编码语义与独立验证，避免两后端同错。

这与 getopt 的模式相同：先把复杂输入变成清楚的中间状态，再由后续步骤处理。架构的价值不是“多了一层”，而是把变化和错误定位到有责任边界的位置。

第 15 节

01:11:08—01:12:33 · 课堂时间

优化与变化：由中间层承接

> **关键判断 · 整理者提炼**
>
> 优化和兼容性应落到负责它们的模块，并以实测成本判断改动是否有效。

中间语言可以承接优化规则：把重复且值不变的计算提到循环外，或按性能分析结果改变表达方式。先测瓶颈，再做 profiling-guided optimization，而不是凭代码看起来长就盲目改写。

整理者说明

讲义补充：常量折叠、公共子表达式共享、LET 或辅助单元格各有适用条件；须分别测首次完整计算、输入变化后的重算和**工作簿体积**。课堂未逐项演示这些优化。

先优化计算图，再决定中间结果放在哪里

- **常量折叠**：固定参数下的定位图案、有限域常量表，在生成时算好。
- **公共子表达式**：相同的纯计算在图中共享一个节点，避免反复粘贴成长公式。
- 公式内复用可用 LET；跨公式共享，或目标版本不支持 LET，可用辅助单元格。

用 Excel 的实际计算成本做选择

- 按依赖关系安排计算；结合复用次数、计算量和目标版本的公式限制，决定内联还是拆格。
- Microsoft 也建议拆解巨型公式、减少重复计算、用辅助单元格保存中间结果。Excel 性能指南
- 分别测量首次完整计算、修改输入后的重算时间；同时观察工作簿体积，避免把缓存成本藏起来。

29 / 47

资料图 · 第 29 页 讲义补充：计算图优化与 Excel 实际成本，包含课堂未展开的具体策略。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 29 · 原始讲义 · 讲义补充：计算图优化与 Excel 实际成本，包含课堂未展开的具体策略。

Excel 版本变化时，可以替换对应算子的**后端实现**，而不从头重做整个二维码算法。这也是其他领域中算子与后端分离的共同思路。

讲义把**责任**进一步写清：版本兼容归目标能力与降级规则；布局移动归地址分配；算法变化归构图或优化；新增输入归输入绑定、拼接、编码与容量。明确的接口使一次变化主要落在负责它的模块中。

资料图 · 第 27 页 讲义补充：四类需求变化分别交给哪个模块。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 27 · 原始讲义 · 讲义补充：四类需求变化分别交给哪个模块。

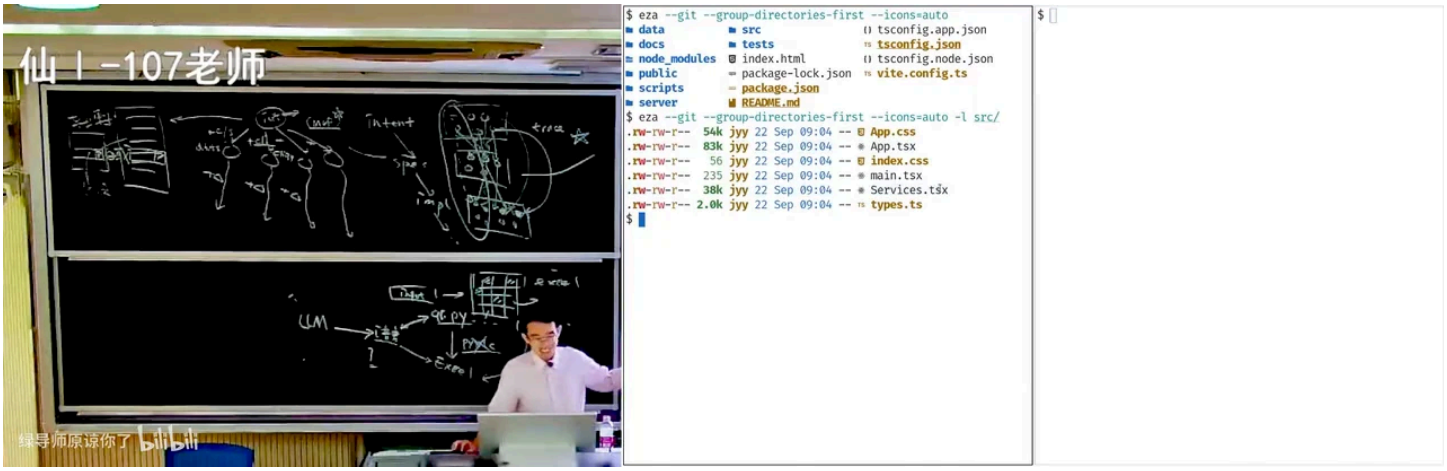
第 16 节

01:12:33—01:17:31 · 课堂时间

大文件能运行，为什么还要重新审视架构

> **关键判断 · 整理者提炼**
>
> 生成项目具备工程机制，不等于其边界适合长期业务变化；还需检查当前状态模型保留和丢失了什么。

回到大型教务系统，讲师先看生成项目的大型 TSX 和 services 文件，再请另一个模型评价。模型识别出**三层单体**、前后端分离，以及乐观锁、幂等、事务审计、统一错误等设计；这些是现场模型评价，笔记没有完整审计该代码库。



视频原帧 74:18 原视频：现场查看生成项目的大型 TSX/services 文件；代码体积不是维护性的充分证据。

视频原帧 01:14:18 · 74:18 原视频：现场查看生成项目的大型 TSX/services 文件；代码体积不是维护性的充分证据。

资料图 · 第 32 页 官方讲义：生成原型能维护的判断带有限定——应拆分并统一规则。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 32 · 原始讲义 · 官方讲义：生成原型能维护的判断带有限定——应拆分并统一规则。

讲师承认生成代码具备一些工程经验，但认为“能维护”和“适合**长期业务演化**”不是同一个判断。把逻辑挤在几个大文件里，后续新增功能可能越来越难预测。

CRUD 将软件理解为现实**状态的投影**：数据库保存学生、教师、院系和选课，业务操作把一个状态改成下一个状态。这个模型本身并非错误，需要进一步考察它保留了什么、丢掉了什么。

第 17 节

01:17:31—01:24:04 · 课堂时间

当前状态掩盖的事实、规则与决定

> **关键判断 · 整理者提炼**
>
> 业务结论需要携带输入、规则版本和批准依据，才能在变化后解释历史并定位需要复核的结果。

数据库规范化用标识符关联实体，减少同一信息多处重复带来的**更新异常**。例如学生改名，其他表引用稳定学号即可，不应靠重复姓名表示身份。

但消除字段冗余并不等于保留**推理依据**。“允许毕业”可能由学分、毕设、培养方案和人工审批计算得出。只保存 true/false，看到结果时无法知道为何成立，成绩或规则变化后也难以定位需要重审的结论。

CRUD 架构的灾难

“当前状态”里，混着事实、规则和决定

- “当前状态” = 历史事实 + 当时适用的规则 + 根据规则作出的决定
  - 有大量单点数据是带有 data dependency 的“结果”
  - Data dependency 意味着**前面改了会摧毁后面**
- 更糟糕的是，这些依赖关系可能没有很好地管理起来 (草台班子嘛)

一个“允许毕业”，究竟依赖什么？

- 课程成绩、课程替代审批、培养方案版本，共同决定学分认定和毕业结论。
  - 成绩更正了：哪些结论要重新审核？哪些已经成为正式决定？
- 表中没有重复字段，也可能丢掉“这个结论是怎么算出来的”。

资料图 · 第 34 页 当前状态混合了历史事实、规则和决定；字段不重复仍可能丢失计算依据。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 34 · 原始讲义 · 当前状态混合了历史事实、规则和决定；字段不重复仍可能丢失计算依据。

官方现行讲义补充，课堂未逐项展开

官方讲义扩展：把追溯落到业务数据里

一次毕业审核可记录输入 ID 和版本或快照、规则版本、结论及**审批依据**。成绩更正后沿依赖找到受影响的审核，正式决定进入复核流程。事件表只是材料，结果对输入的关系仍须明确建模。

PDF物理页 41、44 · 原始资料 ↗

实践：让一个结论带着它的依据

规则与依赖，也成为数据

- 一次毕业审核记录：**输入记录 ID 与版本（或快照）+ 规则版本 + 结论 + 审批依据。**
  - 成绩更正后，沿依赖关系找到受影响的审核；已发布的决定进入复核流程。
- 规则独立版本化：同一批事实可以计算候选方案，并列比较差异。

现实发生的时间，系统知道的时间

- 例：豁免 6 月 1 日生效，6 月 3 日才录入；这两个时间都要保留。
- 因此能分别回答 “6 月 2 日豁免是否已生效” 和 “6 月 2 日系统为何拒绝毕业”。双时态历史
- 这是依赖、决策与时间的建模；事件表之外，还要显式记录结果对输入的依赖。

资料图 · 第 44 页 讲义补充｜结论携带输入/规则/审批依据；区分事件生效与系统获知的时间。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 44 · 原始讲义 · 讲义补充｜结论携带输入/规则/审批依据；区分事件生效与系统获知的时间。

现实世界还会领先于系统：政策已经批准，方案尚未导入；教室不能使用，系统仍按原计划排课；学生急需毕业，软件尚不能表达特例。人工修改当前状态能救急，却可能不断累积无法解释的数据。

资料图 · 第 35 页 讲义补充：把人工例外表达成带批准人和理由的业务过程。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 35 · 原始讲义 · 讲义补充：把人工例外表达成带批准人和理由的业务过程。

需要区分**历史事实**、当时规则以及据此作出的正式决定。业务结论应保留输入与版本、**审批依据**和影响关系，否则让 AI 修当前报错，也补不回过去被丢弃的含义。

第 18 节

01:24:04—01:30:00 · 课堂时间

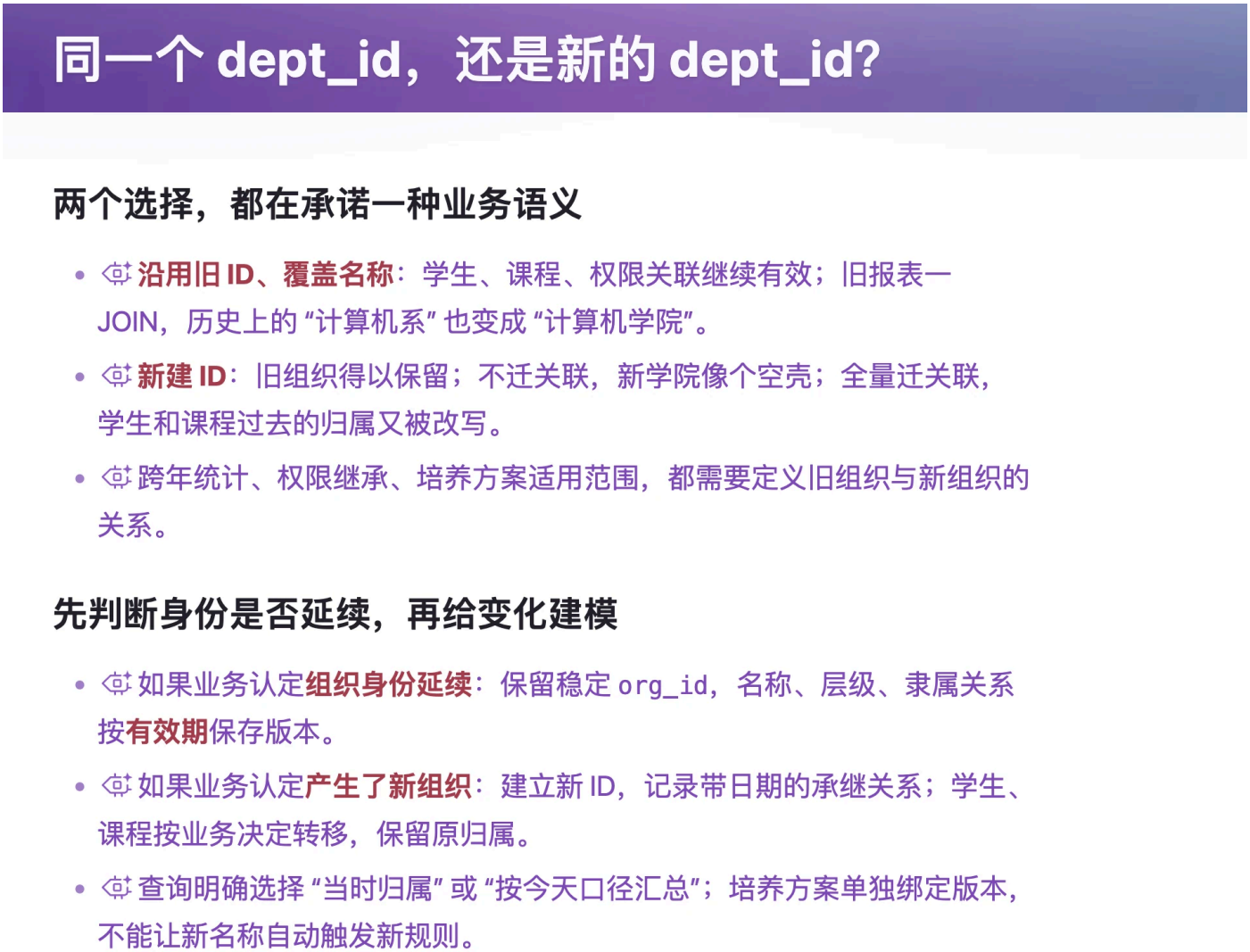
计算机系变学院：身份、属性与历史归属

> **关键判断 · 整理者提炼**
>
> 组织身份是否延续、属性何时有效和历史归属是什么，需要分别建模。

若直接将旧 department 记录名称覆盖为“计算机学院”，历史毕业记录一经关联查询，也会显示新名称，与当年事实不符。若新建对象，现有学生、课程、权限仍指向旧对象，迁移关联又可能改写**历史归属**。

问题不是简单选“改名”或“新 ID”，而是判断**业务身份**是否延续，以及名称、层级和隶属关系在什么时间有效。当前结构下汇总，与查询当时归属，是两种不同口径。

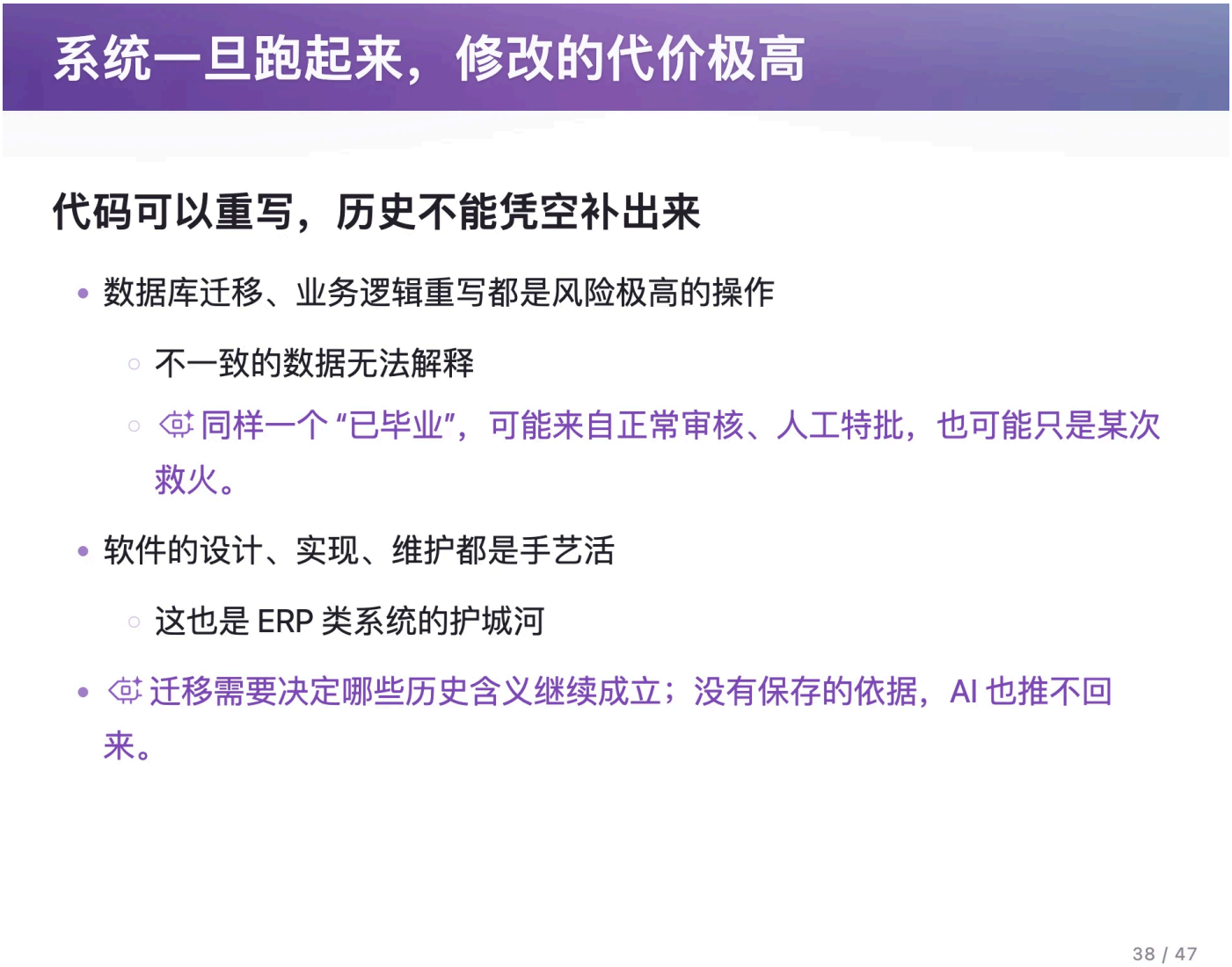
> **整理者说明**
> 讲义补充：身份延续时可保留**稳定 org_id** 并为属性保留**有效期**；产生新组织时创建新 ID、记录带日期的承继关系，同时保留原归属。培养方案应单独版本化，不能随名称变化自动更换。



资料图 · 第 37 页 讲义补充：旧 ID 与新 ID 各自承诺的业务含义，以及按有效期建模的方案。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 37 · 原始讲义 · 讲义补充：旧 ID 与新 ID 各自承诺的业务含义，以及按有效期建模的方案。

代码可以重新生成，但历史依据不能凭空补回。测试环境里的干净数据迁移，与长期运行、混有人工救火数据的生产迁移，风险不同。讲师以过去研究生教务系统在扩招与培养类型变化中难以维护的经历，说明领域知识与持续维护经验的价值。



资料图 · 第 38 页 代码可重写，历史不能凭空补出；这也是复杂 ERP 维护困难的来源。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 38 · 原始讲义 · 代码可重写，历史不能凭空补出；这也是复杂 ERP 维护困难的来源。

第 19 节

01:30:00—01:35:55 · 课堂时间

事件溯源：保留事实，再按需要投影状态

> **关键判断 · 整理者提炼**
>
> 事件溯源保留业务事实并投影状态，但重放仍需处理版本、规则与外部副作用。

讲师从可持久化数据结构引出另一种观察方式：不仅保存当前状态，也可以保留**历史版本**、操作或状态差量。课堂用树的路径复制说明结构共享，也提到日志与版本管理；重点是让过去的变化不再随更新消失。

> **整理者说明**
>
> 整理者校正：数据库 journal、Git、可持久化树和业务事件不是同一种语义结构，不能认为有底层日志就自动完成**事件溯源**。需要保存被业务接受的事实及其必要依据。

把依赖变成 First-Class Citizens

> 如果我们跳出问题看问题：既然都知道 dependency 是问题，为什么不能直接把它们变成 first-class citizens?

数据结构的两面：当前状态与操作历史

- 数据结构可以记录 “当前状态”，也可以记录 “操作历史”
  - 张三选修了《生成式软件工程》
- 状态由 replay 事件序列决定 (语义 = 状态机)
  - 这样，我们就有底气面对语义改变了
  - 当然，还有很多问题要小心处理，比如语义的版本
- 祝贺，你发明了 event sourcing
- 账本的直觉：余额是结果，逐笔记录让我们知道结果从何而来。

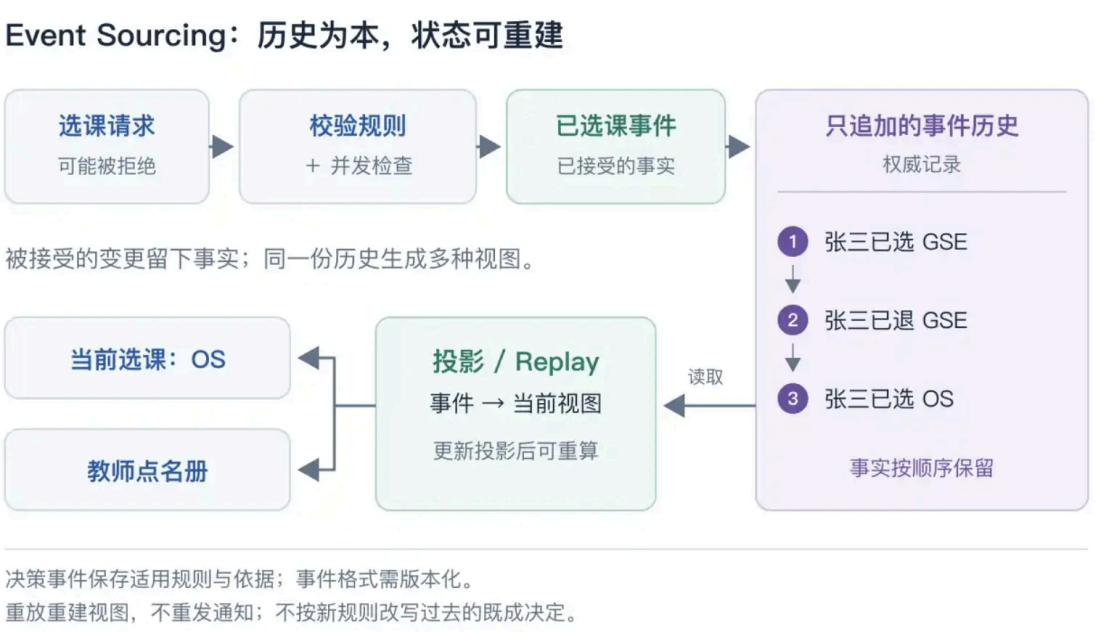
资料图 · 第 39 页 当前状态与操作历史：将依赖与历史变成可表示的对象。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 39 · 原始讲义 · 当前状态与操作历史：将依赖与历史变成可表示的对象。

若退学操作只是删除学生及关联，后来新增“复学”需求时，很难重建以前的事实。若退学等事件已经保存，就可以 replay 到相关历史状态，再据新业务逻辑处理复学。新功能的业务逻辑仍要写，保存历史不会自动替你完成需求。

院系调整同样可以成为带时间的事件：查询毕业时的归属使用当时历史，查询当前情况可以投影与该人相关的后续变化。Event Sourcing（**事件溯源**）就是从历史事件生成状态，而不只原地维护当前快照。

Event Sourcing：从事件历史生成状态



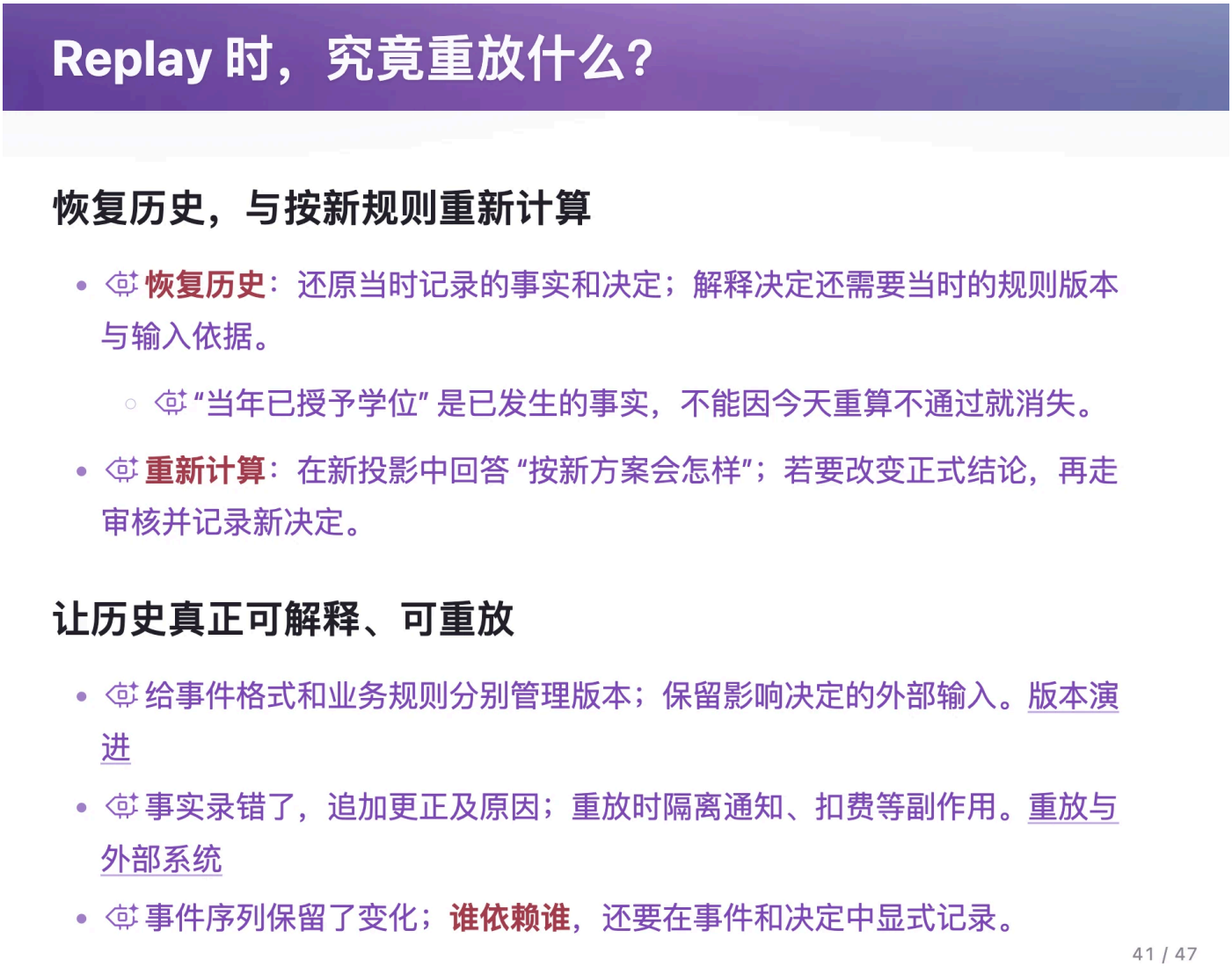
按顺序应用已经接受的事实；新事件增量更新，历史事件支持重建。参见 Event Sourcing。

资料图 · 第 40 页 官方讲义：事件历史生成状态的流程图。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 40 · 原始讲义 · 官方讲义：事件历史生成状态的流程图。

重放性能可以借助快照、缓存及增量更新控制。具体选择仍是 trade-off：既有系统迁到**事件溯源**很困难，提前保留足够历史则会为未来改变规则留下余地。

> **整理者说明**
>
> 讲义补充：事件格式与规则分别版本化；恢复历史和按新规则重算分开，重放不应**重复发通知或扣费**。既有“授予学位”事实不会因新规则计算不通过而自动消失。



资料图 · 第 41 页 讲义补充｜课堂未完整展开：历史恢复、新规则重算、版本与外部副作用。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 41 · 原始讲义 · 讲义补充｜课堂未完整展开：历史恢复、新规则重算、版本与外部副作用。

第 20 节

01:35:55—01:36:50 · 课堂时间

结尾：CQRS、DDD 与回到第一性原理

> **关键判断 · 整理者提炼**
>
> CQRS、DDD 与事件溯源承担不同职责，实际选择还要面对一致性边界与渐进迁移。

讲师在结尾提到 CQRS（Command Query Responsibility Segregation）与 DDD（Domain-Driven Design），并明确 CQRS 与 Event Sourcing 是**独立的选择**。课堂没有逐项讲授这些方法的完整实现。

本课收束到一个问题：面对五十万行教务系统，先跳出当前代码，找到**复杂性的来源**，再选择能承接变化的表示。即使未听过这些术语，也可以从事实、依赖和时间的需求推导出合理设计。

本节及相关知识点保留官方现行讲义结尾扩展内容，并明确区分口述。CQRS 分离接受操作的写模型与提供查询的读模型；DDD 让业务概念与**限界上下文**决定边界；Event Sourcing 决定如何记录和重建历史。三者可组合，却互不等同。

CQRS：读与写，可以有不同的模型

Command Query Responsibility Segregation

- **Command（写）**：表达业务意图，如“申请选课”；检查资格、容量、冲突，然后接受或拒绝。
- **Query（读）**：按使用者组织信息，如学生课表、教师点名册、教务统计。
- 读模型可以针对查询做冗余和预计算，写模型集中维护约束。CQRS

分开之后，要约定如何同步

- 如果异步更新视图：选课已确认，但课表还没刷新，界面应能解释这个间隙。
- 读写模型可以共享进程和数据库；是否使用 Event Sourcing，是另一项选择。实现方式
- Pre-LLM 时代就有这些实践：我们开始分别设计“怎样做决定”和“怎样看结果”。

资料图 · 第 42 页 讲义补充｜CQRS：写端维护资格、容量等约束，读端按使用者组织信息。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 42 · 原始讲义 · 讲义补充｜CQRS：写端维护资格、容量等约束，读端按使用者组织信息。

DDD：让业务概念决定软件边界

Domain-Driven Design

- 开发者与教务人员共同建立业务模型，让“选课”“学分认定”“毕业审批”成为代码中的概念。
- **统一语言**：在一个明确边界内，同一个词具有一致含义。领域模型与语言

“学生”并不只有一种模型

- 选课关心资格与名额；财务关心应收与到账；毕业审核关心认定学分与适用方案。
- 建立**限界上下文**：各自维护自己的模型，通过明确的接口和含义对接。
  - “课程成绩已通过”传给学分认定；能否计入本专业学分，由学分认定的规则决定。
- DDD 安排业务语义和边界；CQRS 分开读写模型；Event Sourcing 选择如何保存历史。三者可以组合。

资料图 · 第 43 页 讲义补充 | DDD：统一语言与限界上下文，不同业务对“学生”的模型可以不同。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 43 · 原始讲义 · 讲义补充 | DDD：统一语言与限界上下文，不同业务对“学生”的模型可以不同。

官方现行讲义补充，课堂未逐项展开

官方讲义扩展：一致性、时间与逐步迁移

最后一个名额不能被两位学生同时占用，容量检查与占位要在一致性边界内完成；异步课表视图可以稍后刷新，但应向用户说明。豁免6月1日生效、6月3日录入，两个时间分别回答业务是否有效与系统当时为何拒绝。迁移先从毕业审核等业务链开始，保留旧系统基线，新旧并行对账，丢失历史不伪造成事件。简单信息维护仍可使用CRUD。

PDF物理页 44、45、46 · 原始资料 ↗

实践：先决定什么必须一起正确

用业务不变量划定一致性边界

- 最后一个名额，两人同时申请：**检查容量与确认占位必须原子地完成。**
  - 事件存储也要检查并发版本；冲突后重新加载、重新判断。并发控制
- 课表、点名册若允许稍后更新，应标明数据更新时间；“选课成功”由写端完成资格检查与占位后确认。

把最容易变化的决定关在边界里

- 培养方案、课程替代、身份映射各有负责模块；调用方依赖明确契约。
- 可以先做模块化单体，把数据归属和模块接口理清，再按独立发布、扩容需求拆服务。
- 信息隐藏的关键是隔离会变化的设计决定。Parnas, 1972

资料图 · 第 45 页 讲义补充｜并发不变量与模块边界，事件追加不自动解决超卖。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 45 · 原始讲义 · 讲义补充｜并发不变量与模块边界，事件追加不自动解决超卖。

实践：让运行中的系统逐步换轨

从一条业务链开始积累可解释的历史

- 先选择“毕业审核”：明确事实来源、规则版本和审批流程，逐步接管其写入入口。
- 旧系统提供切换时的基线快照，明确它的时间与来源；缺失的历史不伪造成事件。
- 一条业务写入只认一个权威来源；其他视图可以由记录的变化重建。

用真实案例验证，再扩大边界

- 新旧审核并行对账：重点解释转专业、补考、课程替代、人工豁免的差异。
- 逐步切换功能，保留故障回退安排；这是 Strangler Fig 式迁移的思路。
- 把追溯与重算用在需要它的业务；教室信息等简单维护仍可采用 CRUD。

资料图 · 第 46 页 讲义补充｜逐步迁移、基线快照、新旧对账与丢失历史的诚实边界。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 46 · 原始讲义 · 讲义补充｜逐步迁移、基线快照、新旧对账与丢失历史的诚实边界。

自测：先回答，再核对

先用自己的话回答，再展开参考答案；答不清楚时，回到对应知识点。

问题 01

为什么 MVP 成功不能证明长期架构合适?

参考答案 · 核对思路

MVP 只走通最重要路径，尚未经历规则变化、例外、历史数据与真实责任；先前决策会持续约束后续实现。

回查第 10 节 →

问题 02

汉诺塔的三种好表示把复杂性放在哪里?

参考答案 · 核对思路

数学规律放在**不变量**和证明；**任务栈**放在待执行任务分解；抽象机器放在栈帧、程序计数器和小步状态转移。

回查第 12 节 →

问题 03

二维码两个后端结果一致就能证明正确吗?

参考答案 · 核对思路

不能。二者可能共享算法错误；还需**独立实现或解码器**。像素比较前须固定编码参数。

回查第 14 节 →

问题 04

为什么只保存 can_graduate=true 不够?

参考答案 · 核对思路

它可能压缩了成绩、方案版本、课程替代与人工批准，失去决定依据后难以解释历史与定位受影响审核。

回查第 17 节 →

问题 05

事件溯源是不是让新需求不用写逻辑?

参考答案 · 核对思路

不是。它保留可重建的历史，新业务仍需逻辑、版本语义和并发处理；重放还需隔离外部副作用。

回查第 19 节 →